

Mocor 平台编译指南

文档版本
发布日期

V1.1
2023-11-17

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档介绍 Mocor 平台编译相关内容，内容涵盖编译环境、Makefile 工作流程、编译命令、模块客制化、第三方编译选项说明、常见使用技巧等。

读者对象

本文档主要适用于 Mocor 平台开发人员。

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.1	2023-11-17	调整文档结构，优化文档内容
V1.0	2020-07-14	第一次正式发布

关键字

编译环境、Makefile、编译命令

目 录

1 编译环境.....	1
1.1 目录结构.....	1
1.2 编译工具.....	2
2 Makefile 工作流程.....	4
2.1 编译脚本.....	4
2.2 Makefile 执行流程.....	4
2.3 Makefile 编译方式.....	6
2.3.1 独立编译.....	6
2.3.2 主编译.....	6
3 编译命令.....	8
3.1 完整编译.....	8
3.2 模块编译.....	9
3.3 更新编译.....	10
3.4 清除编译.....	10
3.5 加快编译速度.....	11
3.6 编译出错暂停.....	11
3.7 生成可执行文件.....	11
3.8 Pclint 扫描.....	12
3.9 编译完成提醒.....	12
3.10 生成编译选项.....	12
4 模块客制化.....	13
4.1 添加或删除模块.....	13
4.2 修改模块.....	14
4.2.1 调整模块目录结构.....	14
4.2.2 添加或删除文件.....	14
4.2.3 修改编译选项.....	14
4.3 创建新项目.....	15
4.4 修改功能开关.....	15
5 第三方编译选项说明.....	16
6 常见使用技巧.....	17
6.1 查看 log 文件.....	17
6.2 常见报错.....	17
6.3 集成 Source Insight 工具.....	17
6.3.1 集成 Pclint 快速定位扫描结果.....	17

6.3.2 集成 ClearCase.....	19
6.4 设置命令行快捷方式.....	21
6.5 设置命令自动填充.....	22

图目录

图 1-1 查看编译器版本	3
图 3-1 Build 操作命令示意图	8
图 6-1 SI 菜单中添加宏	18
图 6-2 设置快捷键	18
图 6-3 扫描结果定位	19
图 6-4 Custom Commands 对话框	19
图 6-5 Menu Assignments 界面	20
图 6-6 checkout 示意图	21

表目录

表 1-1 Build 环境目录说明	2
表 2-1 特殊文件说明	4
表 6-1 Run 命令行可添加的命令	20

1 编译环境

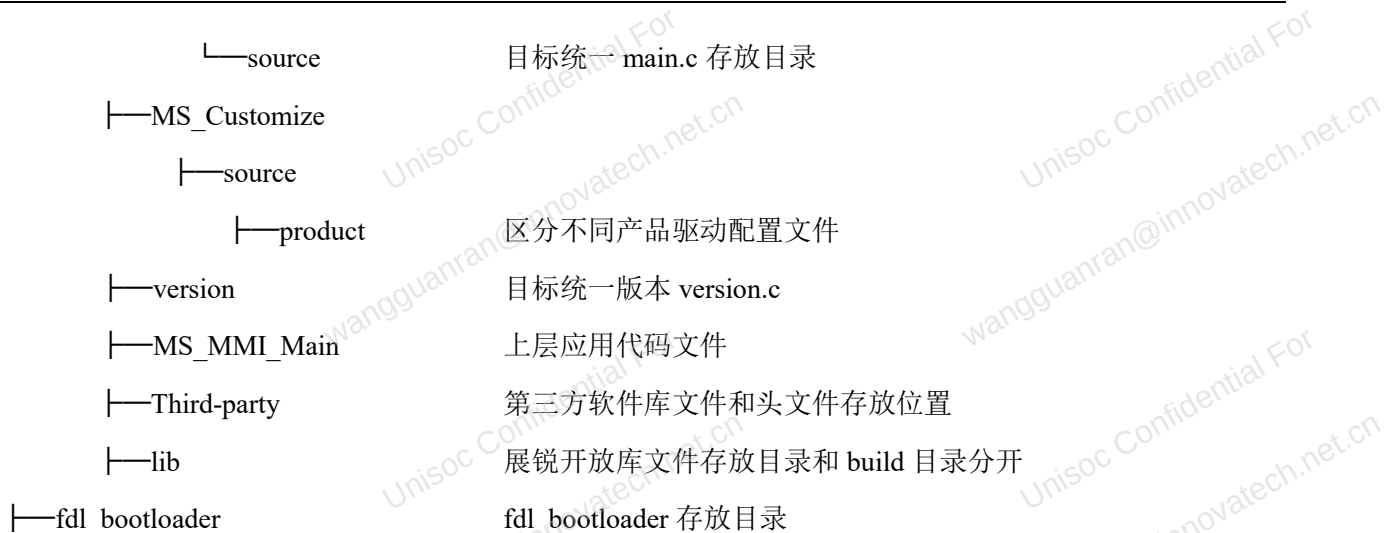
Build 环境中实现了项目开发和维护使用的 GNU 标准应用脚本，基于这些应用脚本和相关 make 工具，可以完成项目全部代码的编译和二进制文件的生成。同时在此环境中执行脚本可以生成其他软件以及开发工具，如 fdl、bootloader、nv_parameter、resource 等。

需要在 Windows Command 环境下执行编译相关命令，每个项目工程编译出来的中间依赖文件都独立存放，因此可以同时开启多个 Windows Command 窗口编译多个不同的项目。

1.1 目录结构

Build 环境目录结构如下所示：

└─platform_all	
└─build	编译结果输出的总目录，此目录下内容在编译过程中生成
└─UWS6133E_1h10_buildidir	与具体项目名称对应，自动生成的输出目录
└─dep	编译依赖文件存放目录
└─img	编译最终结果存放目录，包括 arm bin、fdl、bootloader、resource、nv 等
└─lib	编译模块.a 存放目录
└─log	编译 log 文件目录，包括编译出错信息、告警信息等
└─fdl_bootloader	编译生成的 fdl_bootloader 文件目录
└─res	编译生成的 resource 文件目录
└─nv	编译生成的 nv 文件存放目录
└─tmp	编译生成的临时文件目录
└─obj	模块目标文件存放目录
└─win	自动生成模拟器工程存放目录
└─sml	编译生成的 sml 文件存放目录
└─chip_drv	Source Code
└─MS_Ref	Source Code
└─MSL	协议栈代码目录
└─common	
└─nv_parameters	统一 NV project 存放目录



Build 环境的所有代码和脚本都统一集中在 platform_all 目录下，其中特殊的目录说明如表 1-1 所示。

表1-1 Build 环境目录说明

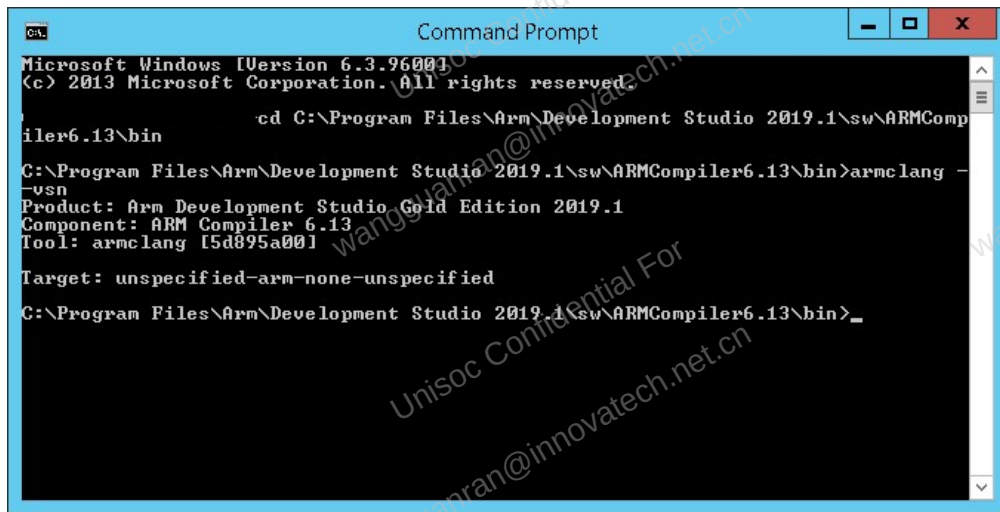
目录	说明
build	build 目录及其下的相关内容在程序编译过程中自动生成，对于特定的项目，build 目录会生成 <i>projectname_build</i> 目录，并在该目录下自动生成 dep、img、lib、log、nv、tmp、obj 等目录。
make	make 目录存放模块对应的编译脚本，子目录和模块一一对应，各个子目录中存放着和目录名称对应的脚本文件。一旦新增或删除模块源文件，相关脚本需要修改。如果新增模块，需要在 make 目录下建立新的模块子目录。
MS_Customize	MS_Customize 目录存放项目相关的 scatter file 文件，文件名称需要和项目名称保持一致。
version	version 目录存放项目相关的 version.c 文件，文件名称需要和项目名称保持一致。

1.2 编译工具

编译工具：Arm® DS 2019.1

Arm® DS 2019.1 中包含的编译器为 armclang，切换到 armclang 的安装路径，使用 “armclang --vsn” 命令可查询编译器的版本号，如图 1-1 所示。

图1-1 查看编译器版本



Makefile.rules 中 ARM_HOME 的定义需要与 Arm® DS 2019.1 中的安装路径保持一致。例如 UWS6131 平台，设置 ARM_HOME 为 Arm® DS 2019.1 中编译工具的路径：

```
ifeq ($(strip $(RVCT_VERSION)), V522)
```

```
    ARM_HOME=C:\Program Files\ARM\Development Studio 2019.1\sw\ARMCompiler6.13
```

当 Arm® DS 的安装路径为自定义时，需要修改 Makefile.rules 中 ARM_HOME 的值为自定义的路径值。此外，编译资源文件的工具需要使用 Excel 软件，编译之前确保 Excel 安装正确。

2 Makefile 工作流程

2.1 编译脚本

platform_all 目录下存放有 Makefile.rules、Makefile.modules、<PLATFORM>.modules、Makefile 以及项目编译脚本，例如 project_UWS6133E_1h10.mk。make 目录的各个子目录下存放有各个模块的 make 脚本（模块编译脚本），例如 aac.mk、app.mk。一般情况下开发人员仅维护项目编译脚本即可，Makefile.rules、Makefile.modules 和 Makefile 不需要修改。

表2-1 特殊文件说明

目录	说明
Makefile.rules	Makefile.rules 中定义了绝大部分与平台、模块、项目无关的公共变量及公共函数，包括编译器的指定、公共编译参数 CFLAGS/AFLAGS 的定义、文件复制/删除、目录创建等系统命令、公共头文件路径、UNISOC 库路径、第三方库文件路径等变量的定义。请根据软件情况进行相应修改。
Makefile.modules	Makefile.modules 定义了真正的编译指令及依赖关系，开发中一般不需要维护该文件。 说明 - 目前部分模块采用了 armcc 编译器（例如 mpeg4_codec），对于这部分模块需要在 Makefile.modules 中申明，具体操作可以参考该文件中已经被申明的模块。 - 分布式编译（Xoreax IncrediBuild），需要 include make/perl_script/modules.pl。
<PLATFORM>.modules	<PLATFORM>.modules 定义了项目需要编译的开放模块、非开放模块、参与链接.a 文件列表等信息。
Makefile.third-party	Makefile.third-party 将第三方逻辑独立成一个文件，便于维护，这个文件会被 include 到各工程.mk 中。
Makefile.verify	Makefile.verify 对 Makefile 中的宏的父子关系进行校验，防止出错。转义的编译宏也在 Makefile.verify 中生成。
Makefile	Makefile 是最上层的总控脚本，该脚本直接处理用户的命令，并执行相关的处理。若无特殊需求，用户不需要维护该文件。

2.2 Makefile 执行流程

Makefile 执行流如下：

Makefile

include Makefile.rules

ARM compiler setting

Common compiler options setting

Bash commands setting

public function setting

makefile location setting

include project_\$(PROJECT).mk

Feature options setting

include Makefile.verify

Parent-child relationships of feature options

Get public MACRO

include Makefile.third-party

Setting third-party modules to variable **3RD_LIBS**

include <PLATFORM.modules>

Set variable:

INDEPENDENT: value specified for external makefile

EXPORTLIBS: value specified for sprd modules list

COMPLIBS_DBG: value specified for modules list of open source

LINKLIBLIST: value specified for all .a list using for armlinking

MKLIBLIST = \$(foreach comp,\$(EXPORTLIBS) \$(COMPLIBS_DBG), \
\$(if \$(findstring \$(comp),\$(INDEPENDENT)),\$(comp).indep, \$(comp).a))

image: \$(MKLIBLIST) \$(3RD_LIBS)

-\$\$(LINK) \$\$(LINKLIBLIST) -o \$(BUILD_DIR)/img/\$(AXFFILENAME)

%.indep:

@\$(MAKE) -f make/\$*/\$.mk -I ./\$(BUILD_DIR)/dep/\$*

%.a:

@\$(MAKE) -f Makefile.modules -I ./\$(BUILD_DIR)/dep/\$* \
-k -r -R COMPONENT=\$*

2.3 Makefile 编译方式

2.3.1 独立编译

采用独立编译的软件模块一般是实现一定的功能，并不一定有.a 文件生成。目前采用独立的 Makefile 编译的软件或者工具模块有 fdl1、fdl2、bootloader、nv_parameter、resource、simulator 等。这些模块或功能对应的“make/module_name/module_name.mk”本身已经是一个完整的 Makefile 文件，将模块名添加到 INDEPENDENT 列表中，make 直接调用执行。

```
# independent compile
```

```
INDEPENDENT= tf_fdl fdl1 fdl2 bootloader nor_fdl preload_app logel mtv_api_mxx fota varm_fw
release_level_d0 mtcoffset
```

编译命令：

```
make PROJECT=projectname MODULES=bootloader
```

如果该 Makefile 中需要使用总工程中宏信息和公共变量定义，请按如下方式包含文件。以下四行是必须的，并且要保持如下顺序。

```
include Makefile.rules
```

```
include ./project_$(PROJECT).mk
```

```
include Makefile.verify
```

```
include Makefile.rules
```

2.3.2 主编译

主编译使用一致的依赖关系生成方法，统一控制编译的行为。绝大部分的模块都是通过主编译方式完成编译，模块对应的“make/module_name/module_name.mk”文件并不是一个完整的 Makefile 文件，一般其中只包含编译需要的宏列表、文件列表、搜索文件路径，其他都由框架自动完成。依赖关系文件存放于 platform_all\build\projectname_build\dep 下。

每个项目工程在编译的时候，脚本会自动将 Feature Options 部分转义成代码中使用的宏传给相应的子模块，在对子模块代码编译时，自动附加上这些宏。转义的规则如下：

- 对功能开关型的 Feature Options，当 Feature Options 为 TRUE 时转义，为 FALSE 时不转义。

```
NANDBOOT_SUPPORT = TRUE ##### Boot From Nandflash Support:
```

```
# Option1:TRUE
```

```
# Option2:FALSE
```

产生转义宏-DNANDBOOT_SUPPORT 传递给各个子模块。

- 对功能枚举型的 Feature Options，将 Feature Options 的名字和价值拼接起来转义成一个宏，对非空的值，同时转义一个 Feature Options 名字的宏。

```
SUBLCD_SIZE = 96X64 ##### Resolution Of Main LCD
```

```
# Option1:NONE #define SUBLCD_SIZE_NONE
```

```
# Option2:96X64 #define SUBLCD_SIZE_96X64
```

产生转义宏-DSUBLCD_SIZE_96X64 和-DSUBLCD_SIZE 传递给各个子模块。

转义函数在 Makefile.rules 中定义，在 Makefile.verify 中调用，转义结果存放在全局变量中。

C_MACRO (for C language)

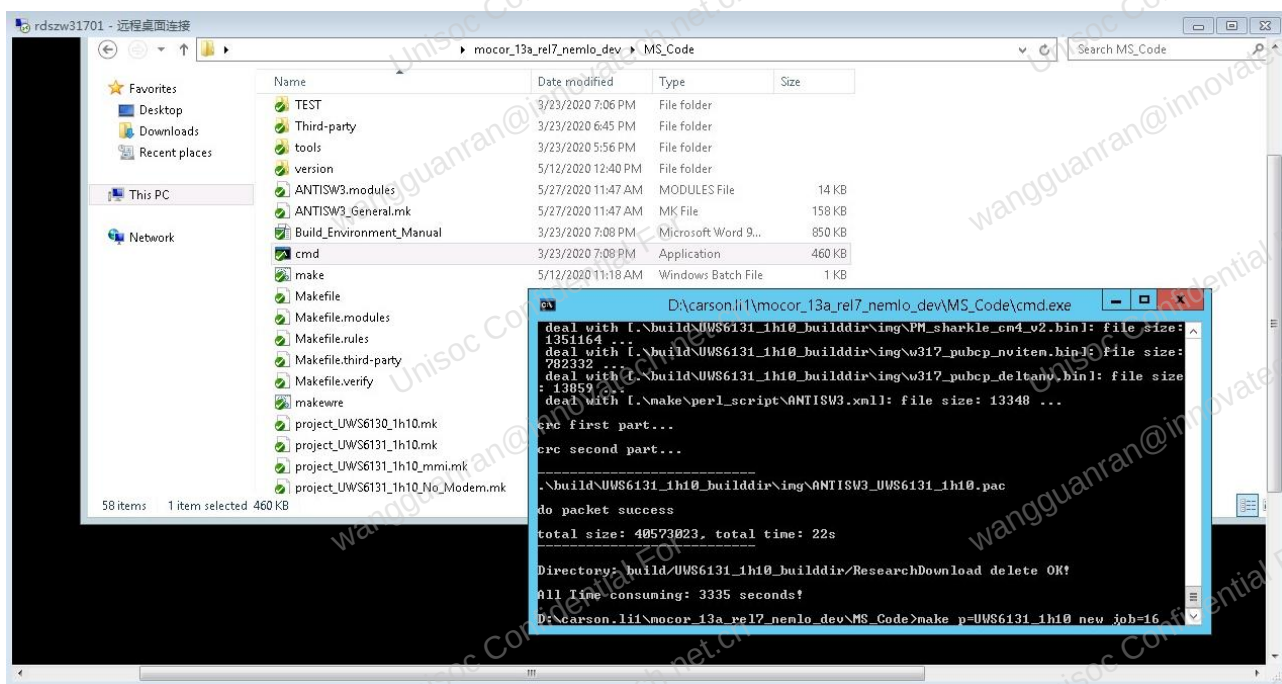
A_MACRO (for Assebley language)

Makefile.rules 文件中变量 `filter-out_macro` 记录了不需要转义成编译宏的 Makefile 变量，其中一些是编译时的命令行参数。如果项目增加了额外的命令行参数，请考虑是否在转义宏时将其过滤。例如，检测 SPRD 宏时可能不通过。

3 编译命令

Build 的所有操作命令都要在 Windows Command 下执行，并且需要在关键脚本的当前路径，即需要在 platform_all 目录下打开 cmd。

图3-1 Build 操作命令示意图



编译项目时，会使用类似“`make project=projectname`”命令，`projectname` 必须和项目编译脚本的名称保持对应关系，否则系统会报以下错误：

Project defined Makefile `project_projectname.mk` is not exist!

如项目编译脚本文件名为 `project_UWS6133E_WC_1h10.mk`，则 `projectname` 的内容为 `UWS6133E_WC_1h10`，因此可以执行如下命令实现编译：

```
make PROJECT= UWS6133E_WC_1h10
```

3.1 完整编译

当项目进行了较大修改，例如公共头文件数据结构发生变化、公共接口参数发生变化、文件更新中时间戳混乱、Makefile 相关脚本被修改等情况，建议使用完整编译，确保重新对项目进行完全编译得到正确的输出结果。

完整编译将清除编译过程中生成的全部目标码和库文件，然后对 *projectname* 指定的项目进行整体编译并在没有错误出现的情况下完成最终的链接，编译过程中会自动生成依赖关系。

通过命令行工具，在项目目录 *platform_all* 下输入如下命令，执行完整编译。

```
make PROJECT=projectname new
```

完整编译命令也会将 *fdl*、*bootloader*、*NV*、*Resource* 同时编译出来。相当于运行了如下 3 个命令：

```
make PROJECT=projectname cleanall
```

```
make PROJECT=projectname MODULES=resource_main
```

```
make PROJECT=projectname update
```

3.2 模块编译

具体模块开发中，需要经常使用模块编译命令来验证模块编译的完整性。同时编译多个模块时，需要用引号将模块包裹起来，单个模块不需要加引号。

通过命令行工具，在项目目录 *platform_all* 下输入对应模块的编译命令即可编译对应模块。

- 编译 *bootloader*

```
make PROJECT=projectname MODULES=bootloader
```

说明

本文中编译命令“PROJECT=”可以替换成“p=”，“MODUL=”可以替换成“m=”。

- 编译 *fdl1*

```
make PROJECT=projectname MODULES="fdl1_nf fdl1"
```

- 编译 *fdl2*

```
make PROJECT=projectname MODULES="fdl2_nf fdl2"
```

输出结果位置：*build/projectname_builddir/fdl_bootloader*。同时会自动将最终的 *bin* 文件复制到 *build/projectname_builddir/img* 下，方便下载到设备中。

也可以根据工程具体情况用一条命令编译出所有目标，例如：

```
make PROJECT="UWS6131_1h10" MODULES="fdl1_nf fdl1 fdl2_nf fdl2 bootloader"
```

- 编译 *NV*

```
make PROJECT=projectname MODULES=nv_parameter
```

输出结果位置：*build/projectname_builddir/nv*。同时会自动将最终的 *bin* 文件复制到 *build/projectname_builddir/img*。

说明

- make/nv_parameter/nv_parameter.mk* 是 *NV* 的 Makefile 文件。
- make/nv_parameter/config_info.nv* 是 *NV* 的配置文件。
- 如果希望单独修改项目某项 *NV* 参数（例如单独修改 *nv_type.xml* 某项），应该在使用 *make* 指令编译出 *NV* 项目文件之后使用 *nvtool* 工具编译保存。
- 编译 *Resource*

```
make PROJECT=projectname MODULES=resource
```

Resource 模块在不同的条件下对不同目录下的源文件进行编译。

- 完整编译时，module 名称由 ANTISW3.modules 中的宏定义自动选择：

```
ifeq ($(strip $(MMI_VER)), MAIN)
INDEPENDENT += resource_main simulator_main simulator_td_main resource_win_main
endif
ifeq ($(strip $(MMI_VER)), STABLE)
INDEPENDENT += resource simulator simulator_td resource_win
endif
```

- 单独编译时，需要根据 MMI_VER 的宏定义手动指定。例如，当前使用 resource_main 文件夹：

```
make PROJECT=projectname MODULES= resource_main
```

编译的中间结构输出位置为 build/projectname_builddir/res，编译最终输出的映像文件将被自动复制到 build/projectname_builddir/img 目录，并删除中间结果输出目录/res。如 UWS6133E_WC_1h10 项目中生成的 resource 映像文件为 build/projectname_builddir/img 目录下的 mmi_res_360x360__adultwatch.bin。

默认情况下编译 Resource 使用了主工程中的 Feature Options 转义宏和 app.mk 模块中的宏，如果需要其他模块的宏，请修改对应的 resource.mk（resource_main.mk）文件。

说明

- 当前不支持多个项目工程的资源并行编译。
- 编译资源文件需要工具脚本支持，如果环境不完整会导致编译过程停止。
- 修改 make\perl_script\ResGen\ResView.ini 中配置项[CustStrTable]，可支持多个字符资源文件，如：

```
[CustStrTable]
l=cust_str_table_01.xls
```

3.3 更新编译

- 更新全部编译结果

```
make PROJECT=projectname
```

或者：

```
make PROJECT=projectname update
```

Makefile 根据已有的依赖关系编译因修改涉及到的相关的模块，并在没有错误出现的情况下完成最终的链接。若不存在依赖关系，Makefile 会在编译文件时自动生成依赖关系。编译时会同时编译出 fdl、bootloader、NV。因为只编译较少的模块文件，可以节省编译时间。

- 更新模块编译结果

```
make PROJECT=projectname MODULES="module1 module2 ..." update
```

对指定模块编译，但编译完成后不会进行自动链接。

3.4 清除编译

- 清除所有编译结果

```
make PROJECT=projectname cleanall
```

将清除编译过程中生成的全部目标码和库文件。

- 清除模块编译结果

通过命令行工具在项目目录 platform_all 下输入命令：

```
make PROJECT=projectname MODULES="module1 module2 ..." clean
```

将清除指定模块的编译输出结果，包含指定模块的目标文件和库文件。适合在指定模块内部出现重大修改或者模块本身 Makefile 脚本被修改，在进行模块编译前使用。

3.5 加快编译速度

- 多线程编译

Build 环境支持多线程编译，可以充分利用多核 CPU 的优势，加快编译速度，必须通过显式的方式指定开启线程数：

```
make p= projectname new job=<模块内开启线程数>
```

```
make p= projectname new job=<模块内开启线程数> -j<同时并发编译的模块数>
```

- <模块内开启线程数>：编译时同时启动线程个数，建议不要超过 CPU 核心数。
- <同时并发编译的模块数>：建议不要超过 4。

例如：

```
make p= UWS6131_1h10 new job=8
```

```
make p= UWS6131_1h10 new job=8 -j4
```

- 分布式编译

Build 环境支持 Xoreax IncrediBuild 分布式编译，可以充分利用网络优势，加快编译速度。在正常的编译指令后增加 ib=1 即可，例如：

```
make p= UWS6131_1h10 new ib=1
```

分布式编译支持依赖关系，当执行 “make p= UWS6133E_WC_1h10 m=app update ib=1” 时，会重新编译所有 App 模块文件。如果代码中的文件位置发生变化，编译时请将涉及的模块先使用 clean 命令清理中间依赖文件，然后再编译，否则可能出现找不到相关文件而无法编译。

3.6 编译出错暂停

默认情况下，编译出错后将继续编译，不会停下来。如果需要 Makefile 支持编译出错暂停功能，请使用 stop 参数编译，并赋值为 1。例如：

```
make p= UWS6131_1h10 stop=1 new
```

遇到异常时，脚本会调用 DOS 下的 Pause 命令，按任意键会继续编译，如果需要强行退出按 “Ctrl+C”。

3.7 生成可执行文件

使用如下命令，可以链接生成相关的可执行文件：


```
make PROJECT=projectname image
```

可在模块修改并编译完成后执行该命令，也可以在第三方库文件更新后执行。

3.8 Pclint 扫描

使用如下命令，可对全部模块代码进行 Pclint 扫描：

```
make PROJECT=projectname pclint
```

也可对指定的模块代码进行 Pclint 扫描，例如：

```
make PROJECT="UWS6133E_WC_1h10 "
```

```
MODULES="app gui ps" pclint
```

3.9 编译完成提醒

使用如下命令，当代码编译完成时，会自动向指定的 IP 地址或机器名发一个 net send 消息，说明该工程编译是否成功。

```
make PROJECT="UWS6133E_WC_1h10 " ip=192.168.0.1
```

```
make PROJECT="UWS6133E_WC_1h10 " ip=pcname
```

说明

这个命令只在生成 image 后才调用，单独编译某个模块不生成 image 时不会调用。

3.10 生成编译选项

Mocor 平台可以通过命令行方式获取宏、搜索路径、文件清单的定义，方便其他工具集成或调试。生成的宏定义文件可在 platform_all\build\<project-name>_builddir\options 中获取。例如：

- 获取整个工程的宏

```
make p=UWS6133E_WC_1h10 options
```

- 获取模块的宏

```
make p=UWS6133E_WC_1h10 m=app options
```

```
make p=UWS6133E_WC_1h10 m="app mmk" options
```

4 模块客制化

Build 环境中的最小编译单位称为模块，Build 系统中将以 platform_all/make 目录下的各个子目录名称作为约定的模块名称，这些子目录和各自的功能模块之间呈一一对应关系。

针对模块或者项目变化，多数情况下开发人员只需要修改模块编译脚本和项目编译脚本。

4.1 添加或删除模块

- 修改相应<PLATFORM>.modules 文件中的变量 EXPORTLIBS 或者 COMPLIBS_DBG，添加或删除指定的模块。

```
EXPORTLIBS = audioservice agc_exp aud_algorithm bsd custom_tone char_lib \
digitalgain dsp_codec dtmf_tone dv eq_exp generic_tone \
layer1 layer1_dsp img_proc display l4 \
mpeg4_codec main ps record rtos \
sfs scaling transsample_rate daps_parser \
voice vsp spml

COMPLIBS_DBG = app custom_drv chip_drv gui mmk Ucom ubot upcc version
aud_dev \
aud_config scp scm atc rebase dc \
sim mmk_sys caf_template caf_interface utility isp_service \
!nor_fdl !nv_parameter
```

模块应该放在 EXPORTLIBS 或 COMPLIBS_DBG 中声明，取决于该模块提供给用户的方式。

- 在 EXPORTLIBS 中声明的模块，表明这些模块将采用库文件的方式提供给用户。
- 在 COMPLIBS_DBG 中声明的模块，表明会提供全部的源代码供用户开发。对于用户自己新增的模块，只能在 COMPLIBS_DBG 变量中申明。

nvparameter、fdl1、fdl2、bootloader 等是一些辅助的文件，只编译出来即可，不需要参与链接。只需要在对应模块前加上“!”符号（例如“!nvparameter !fdl1 !fdl2 !bootloader”），就可以将这些模块排除在项目的链接之外。如果需要在项目中增加新的模块，则只需要将该模块名称列举到这个序列中，用空格和其他模块分隔开即可。

说明

“\”符号后面应该紧跟换行符，不能有空格，序列中最后一个模块不能带有“\”。

- 添加或删除模块对应的 mk 脚本

若有新模块需要添加到 Build 系统中，需要在 platform_all/make 目录下建立对应模块的子目录。同时在该目录下完成模块编译脚本（和目录名称一致以.mk 为后缀结尾），该脚本的主要任务是完成

MCFLAG_OPT、MINCPATH、MSRCPATH、SOURCES 的定义，如果模块中有需要编译汇编文件，还需定义 MASMFLAG_OPT。

- 删除模块

如果有模块需要删除，删除 platform_all/make 目录下对应模块的子目录即可。

4.2 修改模块

4.2.1 调整模块目录结构

如果模块内部存储目录结构或者模块整体目录发生变化，则需要修改模块相应的模块编译脚本和变量 MSRCPATH。以 aac 模块为例：

在 make/aac.mk 中定义了：

```
MSRCPATH = MS_Ref/codec/aac/src MS_Ref/codec/aac/src/decoder
```

这里 MSRCPATH 用来定义该模块源文件的搜索路径，如果模块中源文件的路径发生了变化，就需要重新定义 MSRCPATH，确保模块编译需要的文件都包含在了搜索路径中。

如果头文件路径发生变化，则需要重新定义变量 MINCPATH，以确保模块使用到的头文件完全被包含在头文件定义路径之中。以 aac.mk 为例，MINCPATH 变量定义如下：

```
MINCPATH = MS_Ref/codec/aac/inc MS_Ref/codec/aac/src/decoder \ MS_Ref/codec/aac/src/aac_plus_inc
```

说明

这里 MSRCPATH 和 MINCPATH 的相对路径都是以 platform_all 作为上级目录而言的，所以需要按照这一约定给出源文件或头文件完整的相对路径。

4.2.2 添加或删除文件

如果模块编译文件有添加或删除，需要修改变量 SOURCES，可以将增加的文件直接添加在原有的定义之后，将需要删除的文件直接从定义中去掉。以 aac.mk 为例，其 SOURCES 定义如下：

```
SOURCES = aac_adp.c aac_lc_decoder.c bits.c cfft_tab.c common.c \
decoder.c filtbank.c huffman.c id3_parse.c \
is.c mp4.c mp4atom.c mp4ff.c mp4sample.c mp4util.c ms.c pns.c pulse.c \
specrec.c syntax.c tns.c internal_var.c
```

说明

“\” 在这里表示续行，在 “\” 后不能有空格，最后一个文件后（这里为 internal_var.c）不应该再出现 “\”。

4.2.3 修改编译选项

为了适应不同平台或者产品的需求，模块编译选项会发生变化，这时需要修改变量 MCFLAG_OPT，通过不同的编译选择来满足需要。以 aac.mk 为例，其 MCFLAG_OPT 定义如下：

```
MCFLAG_OPT = -Otime -zc -D_AACARM_
```

“-Otime -zc” 属于该模块需要的编译器特殊选项，宏 “_AACARM_” 属于模块自身的特殊选项。

说明

对于“_AACARM_”这类模块内部定义的编译选项，需要在前面加上“-D”以示为编译选项。

4.3 创建新项目

创建新项目的操作步骤如下：

步骤 1 在 platform_all 目录下创建一个新的项目编译脚本，脚本的命名规则如下：

“project_” + “项目名称” + 文件后缀 “.mk”

例如，项目名称为 abcd，则对应的项目编译脚本名称必须是 project_abcd.mk。

步骤 2 在\platform_all\MS_Customize\source\product\config 目录下创建以 *projectname* 为名的文件夹，初始化所有的项目定制文件，注意修改 make\custom_drv\custom_drv.mk。

步骤 3 修改 project_projectname.mk 中的 PRODUCT_CONFIG 变量值指定为 “projectname”。

步骤 4 在\platform_all\MS_Customize 下创建编译需要的 scatter file 文件。

步骤 5 在\platform_all\Version 下创建对应项目的版本文件（version.c）。

步骤 6 修改\platform_all\make\resource\resource_main.mk 指定资源路径。

步骤 7 修改\platform_all\make\nv_parameter\nv_parameter.mk 指定 NV 配置文件。

----结束

为了加快创建工程文件的过程，可以基于一个已有的工程文件进行修改。复制已有工程文件修改名字，在这个基础上配置想要的工程配置，需注意以下信息：

- 新工程中 PRODUCT_CONFIG 指定的目录是否需要修改，是否需要创建一个新的驱动文件集。
- 新工程中 PRODUCT_BASELIB_DIR 指定的非开放模块.a 目录名是否需要修改。
- 新工程的 NV 是否需要修改，检查 nv_parameter.mk 文件是否满足需求。
- 新工程的 Resource 是否需要修改，检查 resource_main.mk 文件是否满足需要。
- 是否为新工程创建一个新的 scatter file 文件。

4.4 修改功能开关

修改项目功能开关相关的编译选项时请注意：

1. 只有主 Makefile 中的“Feature Option(For User)”区域的宏开关，可以自行开关。
2. 任何除 1 以外的宏开关（包括子 Makefile、代码），用户均不可自行修改。如需要更改，请联系展锐 FAE，由展锐修改宏并重新编译版本。如果自行修改将导致编译错误：

```
Can't return outside a subroutine at ./make/perl_script/sprd_macro_check.pl line XXX
```

3. 不同的项目或平台需要选择不同的第三方库文件参与链接，所以需要修改文件 Makefile.third-party。

5

第三方编译选项说明

第三方软件提供商，请使用如下的编译选项来编译代码以配合展锐 Mocor 平台。

```
CFLAGS = --tool_variant=ult --target=arm-arm-none-eabi -march=armv8-a+nofp+nosimd -mcpu=cortex-a53 -  
fshort-wchar -D_DEBUG -D__MODULE__="__FILE__" -Ofast -Os -mno-unaligned-access -g
```

```
AFLAGS = --cpu=8-A.32 -g
```

6 常见使用技巧

6.1 查看 log 文件

Build 相关的屏幕输出同时也被重定向输出到了各个模块的编译 log 文件中，一旦系统报告链接错误：

Could not open file build/rocky_dvb_buildddir/lib/module_n.a

则表明模块 `module_n` 没有正确编译完成，可以查看该模块的相关 log，并查询“error”或者“rule”相关的内容，查明出错的原因。

log 位置：platform_all/build/ *projectname* _buildddir/log

6.2 常见报错

- Makefile 报错：No rule to make target 'XXX', needed by 'yyy'

例如：make[1]: *** No rule to make target 'guianim.o', needed by 'gui.a'.

此处报错说明编译 `gui.a` 时，无法生成 `guianim.o`。原因是没有搜索到生成 `guianim.o` 所使用的源文件，这个文件可能是 `guianim.c`、`guianim.cpp`、`guianim.s` 等。

根源有可能为：

- .mk 中宏使用不对。
- 源文件不存在。
- Makefile 中源文件的搜索路径 `MSRCPATH` 不完整导致 make 找不到源文件。

- Makefile 报错：Can't return outside a subroutine at ./make/perl_script/sprd_macro_check.pl line XXX

只能修改工程文件中 Feature Options (For User) 部分的 Feature 值，否则 make 会给出错误信息并停止编译。

此功能在 Makefile 文件目标 `builddirs` 中通过调用 `./make/perl_script/sprd_macro_check.pl` 脚本实现。

6.3 集成 Source Insight 工具

6.3.1 集成 Pclint 快速定位扫描结果

Source Insight (SI) 中集成 Pclint 快速定位扫描结果的方法如下：

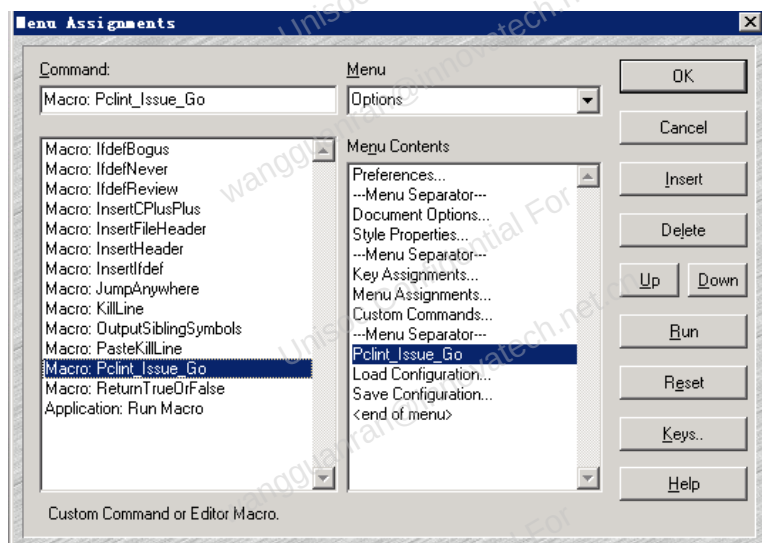
步骤 1 确保 `platform_all\make\make_cmd\PCLINT.EM` 已加入 SI 的工程文件中。

步骤 2 确保各模块的 Pclint 扫描结果文件 `aloe.xml` 已加入 SI 的工程文件中。

步骤 3 在 SI 的菜单“Options > Menu Assignments”将宏 `Pclint_Issue_Go` 添加到菜单中。例如添加到

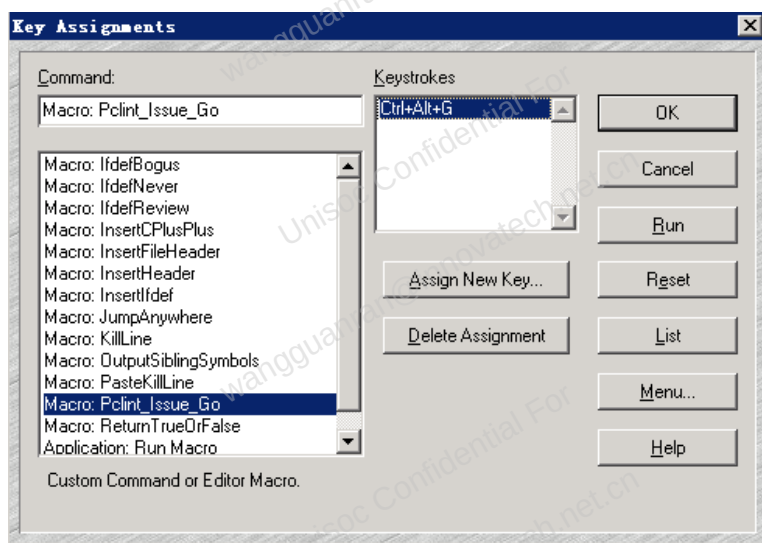
Options 菜单中，如图 6-1 所示。

图6-1 SI 菜单中添加宏



步骤 4 在 SI 的菜单 “Options > Key Assignments” 为宏 Pclint_Issue_Go 设置快捷键。例如设置快捷键为 “Ctrl+Alt+G”，如图 6-2 所示。

图6-2 设置快捷键



说明

一般来说步骤 3 和步骤 4 选择一种即可，并且仅需设置一次。

步骤 5 在 SI 中打开具体模块的 aloa.xml 文件，将鼠标放置到具体的 issue 行，按快捷键或执行菜单命令 Pclint_Issue_Go，就会自动打开具体有问题的文件，并跳转到该行，如图 6-3 所示。

图6-3 扫描结果定位

```
<lintanalysis>
<issue file = "MS_Customize\source\product\driver\lcd\lft_hx8357.c" line = "76" number = "785" desc = "Too few initi
<issue file = "MS_Customize\source\product\driver\norflash\spanion_vs_drv.c" line = "124" number = "527" desc = "
<issue file = "MS_Customize\source\product\driver\norflash\normcp_spec.c" line = "195" number = "413" desc = "Lik
<issue file = "MS_Customize\source\product\driver\norflash\normcp_spec.c" line = "195" number = "413" desc = "Lik
<issue file = "MS_Customize\source\product\driver\norflash\normcp_spec.c" line = "198" number = "413" desc = "Lik
<issue file = "MS_Customize\source\product\driver\norflash\normcp_spec.c" line = "201" number = "413" desc = "Lik
</lintanalysis>
```

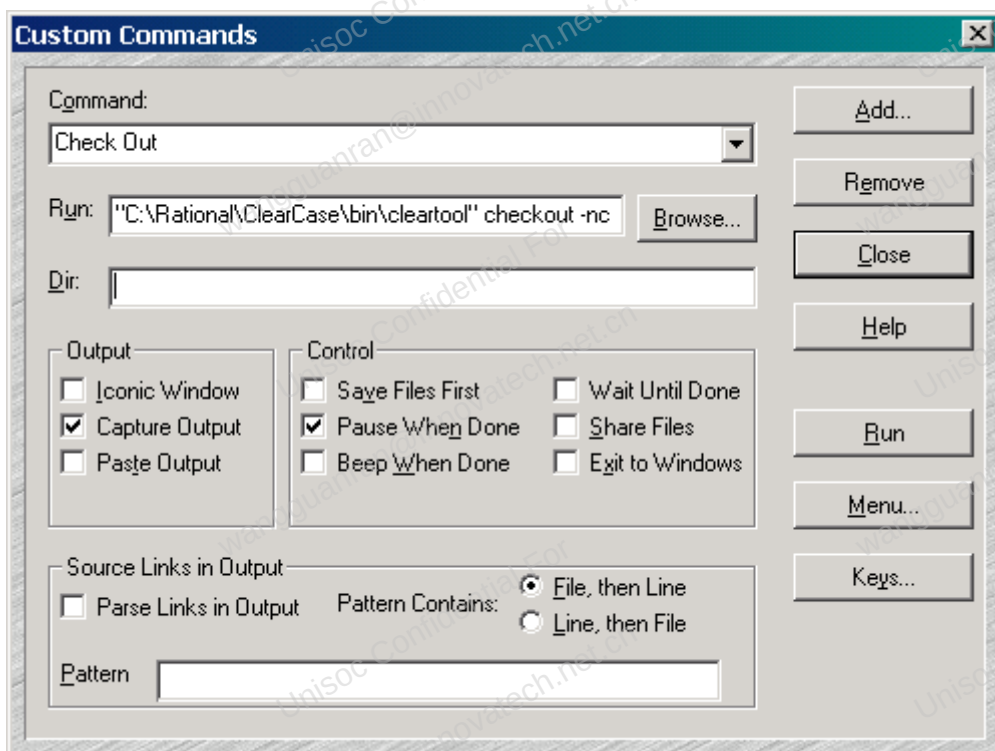
----结束

6.3.2 集成 ClearCase

本节以 Check Out 为例进行介绍，具体配置过程如下：

- 步骤 1 选择 SI 的菜单 Option 中的“Custom Commands”。
- 步骤 2 在弹出的对话框中的 Command 列表中选择“Check Out”，或者点击 Add 自行命名，如图 6-4 所示。

图6-4 Custom Commands 对话框



- 步骤 3 在 Run 命令行中添加命令：

```
"C:\Rational\ClearCase\bin\cleartool" checkout -nc %f
```

如果不是使用 ClearCase 的默认安装路径，“C:\Rational\ClearCase\bin\cleartool”需要修改为指定的安装路径。Run 命令行可添加的命令参考表 6-1。

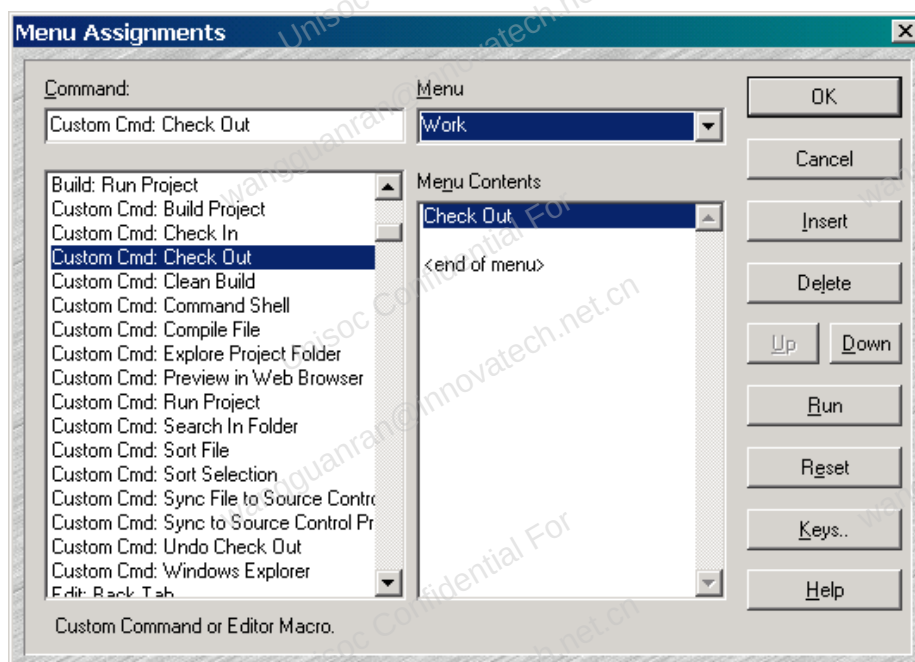
表6-1 Run 命令行可添加的命令

操作	命令
Check In	"C:\Rational\ClearCase\bin\cleardlg" /checkin %f
Check Out	"C:\Rational\ClearCase\bin\cleartool" checkout -nc %f
Undo Check Out	"C:\Rational\ClearCase\bin\cleartool" uncheckout -rm %f
Version Tree	"C:\Rational\ClearCase\bin\cleartool" lsvtree -graphical -all %f 说明 使用此命令时可以不勾选“Capture Output”和“Pause When Done”。
Find	"C:\Rational\ClearCase\bin\cleartool" find %d -element brtype(your_branch_name) -print 查找当前目录及子目录下自己分支修改的文件。
Update File	"C:\Rational\ClearCase\bin\cleartool" update %f
Update View	"C:\Rational\ClearCase\bin\cleartool" update -g
Compare Preversion	"C:\Rational\ClearCase\bin\cleartool" diff -g -pre %f

步骤 4 勾选 Output 中的“Capture Output”和 Control 中的“Pause When Done”，这样才能看到命令执行的结果。

步骤 5 将命令加入菜单项，点击 Menu，在 Menu 下拉框中选择“Work”等进行添加。再选择“Custom Cmd: Check Out”，单击 Insert 按钮添加，如图 6-5 所示。

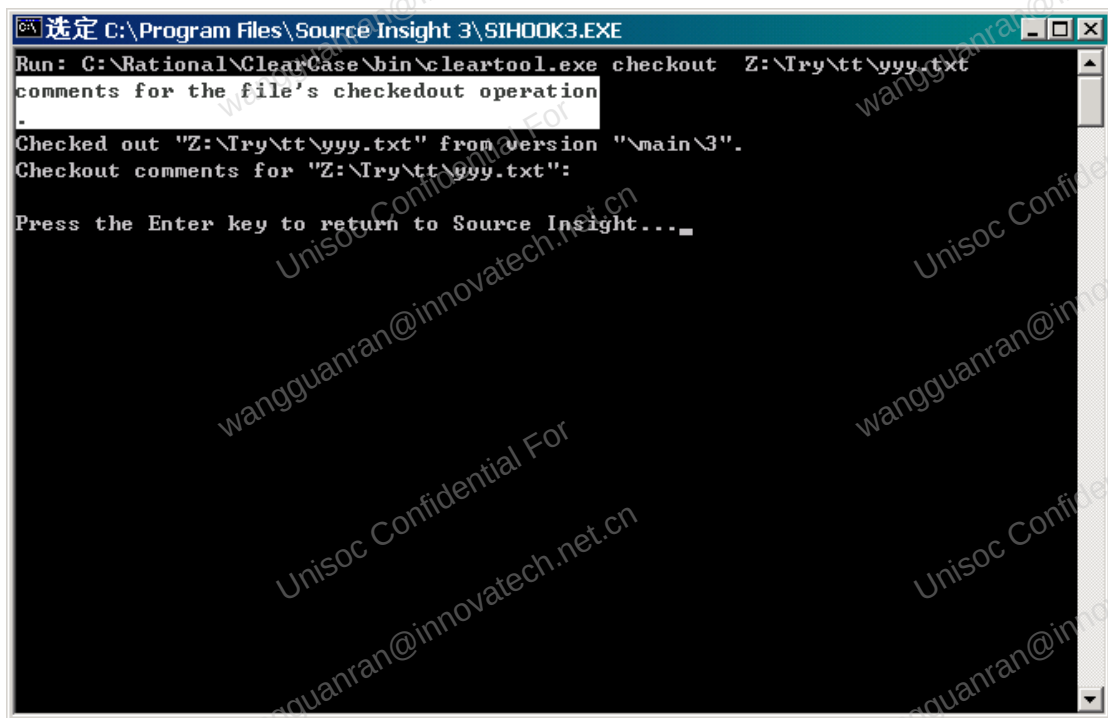
图6-5 Menu Assignments 界面



步骤 6 checkout 操作。

在 SI 中打开视图上的源文件，单击 work 菜单下的“Check Out”，对文件进行 checkout，会弹出 checkout 的操作，如图 6-6 所示。

图6-6 checkout 示意图



其中，“comments for the file's checkedout operation”是 checkout 时需要添加的注释信息，输入完成后回车，然后再输入一个点“.”，再次回车，完成输入。

----结束

6.4 设置命令行快捷方式

设置鼠标右键命令行快捷方式，将如下文本保存为 CommandPrompt.reg 文件，然后双击导入到注册表即可。

Windows Registry Editor Version 5.00

[HKEY_CLASSES_ROOT\Folder\shell\CommandPrompt]

@="Command Prompt"

[HKEY_CLASSES_ROOT\Folder\shell\CommandPrompt\Command]

@="CMD /k cd %1"

6.5 设置命令自动填充

设置 DOS 下输入命令时用 Tab 键进行命令自动填充，将如下文本保存为 DOSTAB.reg 文件，然后双击导入到注册表即可。

Windows Registry Editor Version 5.00

[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Command Processor]

"CompletionChar"=dword:00000009